

1 Mini Project Title

Tweet Sentiment Analysis System

Technology Used: Python, Natural Language Processing (NLP), TF-IDF, Logistic Regression, Pandas, NLTK and Scikit-learn.

The project is designed to analyse tweets and automatically classify them into positive or negative sentiment categories using text preprocessing, Information Retrieval techniques and machine learning.

2 Problem Statement

Social media platforms generate a large amount of textual information in the form of tweets and user opinions. Manually analysing this large volume of text is time-consuming and difficult.

The objective of this mini project is to develop a Tweet Sentiment Analysis System that automatically analyses the textual content of tweets and classifies them into positive or negative sentiment.

The system uses Natural Language Processing techniques to clean the tweet text, an Information Retrieval technique called Term Frequency-Inverse Document Frequency (TF-IDF) to represent the text, and a Logistic Regression machine learning classifier to predict the sentiment.

3 Objectives

The main objectives of the project are:

1. To develop a system for automatic sentiment analysis of tweets.
2. To preprocess and clean raw tweet data.
3. To remove unnecessary information such as URLs, mentions, punctuation and stop words.
4. To convert textual data into numerical feature vectors using TF-IDF.
5. To train a machine learning model for sentiment classification.
6. To classify tweets into positive and negative sentiment.
7. To evaluate the performance of the trained model using accuracy, precision, recall and F1-score.
8. To provide a function that can predict the sentiment of a new tweet.

4 Hardware and Software Requirements

4.1 Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM
- Minimum 2 GB free storage
- Modern processor
- Internet connection for downloading datasets and libraries

4.2 Software Requirements

Software / Library	Purpose
Python	Programming language
Jupyter Notebook	Development environment
Pandas	Dataset loading and processing
NumPy	Numerical operations
NLTK	Text preprocessing
Scikit-learn	TF-IDF and machine learning
Matplotlib	Data visualization
Seaborn	Confusion matrix visualization

Table 1: Software and libraries used

5 Dataset Description

The dataset used for this project contains tweets along with their corresponding sentiment labels.

The dataset contains two main columns:

- **sentence**: Contains the textual content of the tweet.
- **sentiment**: Contains the sentiment label.

The sentiment labels used in the dataset are:

- **0 – Negative**
- **1 – Positive**

The complete training dataset contained approximately 1.87 million records. Since processing the complete dataset was unnecessary for the mini project and would require additional computational time, a random sample of 50,000 tweets was selected.

The selected dataset was divided into:

- 80% training data

- 20% testing data

The dataset contains examples of informal language, abbreviations, and other characteristics commonly found in social media text.

6 Methodology / Algorithm

The proposed system follows the following steps:

1. Load the tweet dataset.
2. Select a suitable subset of tweets for model training.
3. Clean and preprocess the tweet text.
4. Remove URLs, user mentions, punctuation and special characters.
5. Convert all text to lowercase.
6. Remove common English stop words.
7. Apply TF-IDF feature extraction.
8. Split the dataset into training and testing sets.
9. Train a Logistic Regression classifier.
10. Predict the sentiment of the test tweets.
11. Calculate accuracy, precision, recall and F1-score.
12. Generate a confusion matrix to analyse the classification results.
13. Use the trained model to predict the sentiment of new tweets.

6.1 Text Preprocessing

Tweet text contains information such as URLs, mentions, hashtags, punctuation and unnecessary words. These elements are removed before feature extraction.

For example:

Original:

I absolutely LOVE this movie!!! @user #awesome

After preprocessing:

absolutely love movie awesome

The following operations are performed:

- Lowercase conversion
- URL removal

- Mention removal
- Hashtag symbol removal
- Punctuation removal
- Special character removal
- Stop word removal
- Extra whitespace removal

6.2 TF-IDF Feature Extraction

TF-IDF stands for Term Frequency-Inverse Document Frequency. It is an important Information Retrieval technique used to represent documents as numerical vectors.

The TF-IDF value is calculated using:

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

where $TF(t, d)$ represents the frequency of a term in a document and $IDF(t)$ represents the importance of the term across the complete collection of documents.

In this project, the TF-IDF vectorizer was configured to use a maximum of 10,000 features. Both unigrams and bigrams were considered.

For example:

Unigrams:

good
movie
excellent

Bigrams:

good movie
very good
excellent movie

The resulting feature matrix contained:

50,000 tweets × 10,000 TF-IDF features

6.3 Logistic Regression Classification

Logistic Regression was used to classify tweets into positive and negative sentiment classes.

The TF-IDF vectors are given as input to the Logistic Regression model. The model learns patterns associated with the two sentiment classes and predicts the sentiment of previously unseen tweets.

7 Practical Implementation

The project was implemented using Python in Jupyter Notebook.

7.1 Importing Libraries

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import re
6 import nltk
7
8 from nltk.corpus import stopwords
9
10 from sklearn.model_selection import train_test_split
11 from sklearn.feature_extraction.text import TfidfVectorizer
12 from sklearn.linear_model import LogisticRegression
13
14 from sklearn.metrics import (
15     accuracy_score,
16     precision_score,
17     recall_score,
18     f1_score,
19     classification_report,
20     confusion_matrix
21 )
```

7.2 Loading the Dataset

```
1 train_df = pd.read_csv(
2     "train_data.csv",
3     engine="python",
4     on_bad_lines="skip"
5 )
6
7 test_df = pd.read_csv(
8     "test_data.csv",
9     engine="python",
10    on_bad_lines="skip"
11 )
12
13 train_df = train_df.sample(
14     n=50000,
15     random_state=42
16 ).reset_index(drop=True)
```

7.3 Tweet Preprocessing

```
1 stop_words = set(stopwords.words("english"))
2
3 def clean_tweet(text):
4     text = str(text).lower()
```

```
5
6     text = re.sub(
7         r"http\S+|www\S+|https\S+",
8         "",
9         text
10    )
11
12    text = re.sub(r"@w+", "", text)
13
14    text = re.sub(r"#", "", text)
15
16    text = re.sub(
17        r"[^a-zA-Z\s]",
18        "",
19        text
20    )
21
22    text = re.sub(
23        r"\s+",
24        " ",
25        text
26    ).strip()
27
28    words = text.split()
29
30    words = [
31        word for word in words
32        if word not in stop_words
33    ]
34
35    return " ".join(words)
36
37
38    train_df["clean_text"] = (
39        train_df["sentence"].apply(clean_tweet)
40    )
```

7.4 TF-IDF Implementation

```
1    tfidf = TfidfVectorizer(
2        max_features=10000,
3        ngram_range=(1, 2),
4        min_df=2,
5        max_df=0.95
6    )
7
8    X = tfidf.fit_transform(
9        train_df["clean_text"]
10    )
11
```

```
12 y = train_df["sentiment"]
13
14 print("TF-IDF matrix shape:", X.shape)
```

The output obtained was:

TF-IDF matrix shape: (50000, 10000)

7.5 Train-Test Split

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X,
3     y,
4     test_size=0.20,
5     random_state=42,
6     stratify=y
7 )
```

Therefore, 40,000 samples were used for training and 10,000 samples were used for testing.

7.6 Model Training

```
1 model = LogisticRegression(
2     max_iter=1000,
3     random_state=42
4 )
5
6 model.fit(X_train, y_train)
```

7.7 Prediction and Evaluation

```
1 y_pred = model.predict(X_test)
2
3 accuracy = accuracy_score(y_test, y_pred)
4 precision = precision_score(y_test, y_pred)
5 recall = recall_score(y_test, y_pred)
6 f1 = f1_score(y_test, y_pred)
7
8 print("Accuracy:", accuracy)
9 print("Precision:", precision)
10 print("Recall:", recall)
11 print("F1 Score:", f1)
12
13 print(classification_report(
14     y_test,
15     y_pred,
16     target_names=["Negative", "Positive"]
17 ))
```

7.8 Confusion Matrix

```
1 cm = confusion_matrix(y_test, y_pred)
2
3 plt.figure(figsize=(7, 5))
4
5 sns.heatmap(
6     cm,
7     annot=True,
8     fmt="d",
9     cmap="Blues",
10    xticklabels=["Negative", "Positive"],
11    yticklabels=["Negative", "Positive"]
12 )
13
14 plt.title("Confusion Matrix")
15 plt.xlabel("Predicted Sentiment")
16 plt.ylabel("Actual Sentiment")
17
18 plt.show()
```

7.9 New Tweet Prediction

The trained model can also analyse a new tweet.

```
1 def predict_sentiment(tweet):
2
3     cleaned = clean_tweet(tweet)
4
5     vector = tfidf.transform([cleaned])
6
7     prediction = model.predict(vector)[0]
8
9     probability = model.predict_proba(vector)[0]
10
11     if prediction == 1:
12         sentiment = "Positive"
13         confidence = probability[1]
14     else:
15         sentiment = "Negative"
16         confidence = probability[0]
17
18     print("Tweet:", tweet)
19     print("Sentiment:", sentiment)
20     print(f"Confidence: {confidence:.2%}")
```

Example:

```
1 predict_sentiment(
2     "I absolutely love this movie! It was amazing!"
3 )
```

The system predicts the sentiment of the supplied tweet and displays the corresponding confidence.

7.10 Performance Results

The performance of the model is evaluated using four major metrics.

Metric	Value
Accuracy	XX.XX%
Precision	XX.XX%
Recall	XX.XX%
F1 Score	XX.XX%

Table 2: Model performance

The confusion matrix generated during the practical implementation can also be included below.

8 Conclusion

The Tweet Sentiment Analysis System was successfully developed using Python, Natural Language Processing, Information Retrieval and Machine Learning techniques.

The system preprocesses raw tweets by removing unnecessary textual information and then uses TF-IDF to convert the cleaned tweets into numerical feature vectors. Logistic Regression is used to classify the tweets into positive and negative sentiment categories.

The model was evaluated using accuracy, precision, recall and F1-score. A confusion matrix was also generated to analyse the classification performance.

The project demonstrates that traditional Information Retrieval techniques such as TF-IDF can be effectively combined with machine learning for automatic sentiment analysis of social media text. The system can also be extended in the future using larger datasets, additional sentiment classes and advanced NLP models such as transformer-based models.